

Laboratorio #3 — Redes Neuronales Recurrentes y LSTM

CC3092 - Deep Learning y Sistemas Inteligentes (respuestas con base en el notebook)

2. Exploración y preparación de los datos

¿Cuántas observaciones y cuántas clases tiene el dataset? ¿Las clases están balanceadas?

El split `train` tiene 25,000 observaciones y el split `test` tiene 25,000 observaciones (además existe un split `unsupervised` de 50,000 reseñas sin etiqueta que no se usa en este laboratorio). Tiene 2 clases: 0 = negativo y 1 = positivo. Las clases están **perfectamente balanceadas**: 12,500 reseñas por clase tanto en `train` como en `test` (`Counter({0: 12500, 1: 12500})` en ambos casos). Esto hace que `accuracy` sea una métrica confiable por sí sola, aunque igualmente se reportan `precision/recall/F1`.

¿Cuál es la longitud de las reseñas? Mínimo, máximo, promedio, distribución.

Longitud en tokens (tokenización propia, ver más abajo), calculada sobre `train`:

- Mínimo: 10 tokens
- Máximo: 2,462 tokens
- Promedio: 232.4 tokens
- Mediana: 173.0 tokens
- Percentil 75: 282.0 · Percentil 90: 455.0 · Percentil 95: 596.0 · Percentil 99: 906.0

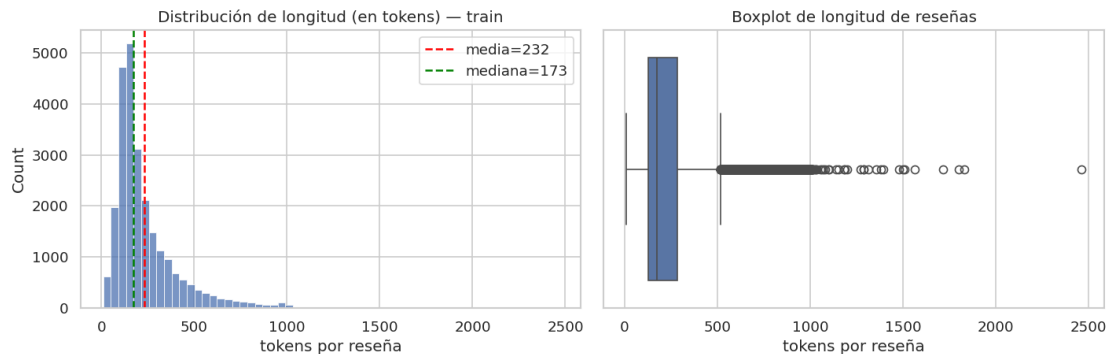


Figura 1: Distribución de longitud de reseñas (train)

La distribución está sesgada a la derecha: la mayoría de reseñas tiene entre 60 y 300 tokens, pero hay una cola larga de reseñas muy extensas (hasta más de 1,000 tokens).

¿Qué estrategia de tokenización usarán? ¿Qué tamaño de vocabulario obtienen y cómo manejarán las palabras fuera de vocabulario?

Tokenización simple basada en expresiones regulares: se pasa el texto a minúsculas, se remueven las etiquetas HTML `<br />` y se extraen secuencias de letras/apóstrofes como tokens (`[a-z']+`). No se usa un tokenizador de subpalabras (BPE/WordPiece) porque el enunciado pide trabajar con `nn.Embedding` + capas recurrentes clásicas, no con un modelo tipo Transformer preentrenado.

El vocabulario se construye **únicamente a partir del split de entrenamiento** (evitando fuga de información de validación/test), limitado a un máximo de 20,000 palabras (`VOCAB_MAX_SIZE=20000`) y descartando palabras con frecuencia menor a 2 (`MIN_FREQ=2`). El tamaño de vocabulario resultante es de **20,000 tokens** (incluye `<pad>` en

el índice 0 y <unk> en el índice 1). Cualquier palabra fuera de este vocabulario (por baja frecuencia en train, o por aparecer solo en validación/test) se mapea al token especial <unk> (OOV).

¿Por qué es necesario aplicar padding y/o truncamiento? ¿Qué longitud máxima eligieron y por qué?

Las reseñas tienen longitud variable, pero para procesarlas en batches con operaciones matriciales (tanto en el MLP como en la RNN/LSTM) todas las secuencias de un mismo batch deben tener la misma longitud. El **padding** rellena las secuencias más cortas que la máxima del batch con el token <pad> (índice 0) usando `pad_sequence`. El **truncamiento** fija una longitud máxima `MAX_LEN` y corta las reseñas más largas, para acotar el costo computacional y de memoria (sin esto, una sola reseña de 1,500 tokens dominaría el costo del batch completo). Adicionalmente, en la RNN/LSTM se usa `pack_padded_sequence` para que el cómputo recurrente no pierda tiempo (ni aprenda) sobre las posiciones de padding.

Con base en la distribución de longitudes (mediana ~ 170, percentil 90 ~ 440), se eligió `MAX_LEN = 256` para la búsqueda principal de hiperparámetros: cubre la mayoría de reseñas casi completas sin disparar el costo computacional por la cola larga de reseñas extremadamente extensas. En la sección 4.1 se explora explícitamente el efecto de usar `MAX_LEN` mucho menor (50) y mucho mayor (450).

Ejemplos del dataset (texto, etiqueta, longitud en tokens):

Texto (primeros 200 caracteres)	Etiqueta	Longitud (tokens)
Arguably this is a very good “sequel”, better ...	positivo	171
It’s a good thing I didn’t watch this while i ...	negativo	119
This has to be the worst movie I have seen. Ma...	negativo	63
“Scarface” has a major cult following even now...	positivo	511
Being born in the 1960’s I grew up watching th...	negativo	166

División en entrenamiento, validación y test

El dataset original solo trae `train/test`. Se creó el split de validación separando un 10% del split `train` de forma estratificada por clase (mezclando primero, porque el split original viene ordenado por etiqueta). El split `test` original queda intacto y solo se usa una vez, al final. Resultado: **train = 22,500** (11,250/11,250), **val = 2,500** (1,250/1,250), **test = 25,000** (12,500/12,500).

3. Investigación: capas de PyTorch para RNN y LSTM

nn.Embedding — Tabla de *lookup* entrenable que mapea índices de tokens (enteros) a vectores densos de dimensión fija; reemplaza a un one-hot + matriz densa, mucho más eficiente en memoria. Parámetros relevantes: `num_embeddings` (tamaño del vocabulario), `embedding_dim`, `padding_idx` (índice cuyo vector se fija en cero y no recibe gradiente, típicamente el de <pad>).

nn.RNN — Red recurrente “vanilla”: en cada paso de tiempo combina la entrada actual con el estado oculto anterior mediante una transformación lineal + no linealidad (`tanh` o `relu`), produciendo un nuevo estado oculto. Sufre de *vanishing/exploding gradient* en secuencias largas. Parámetros relevantes: `input_size`, `hidden_size`, `num_layers`, `nonlinearity` (`tanh/relu`), `batch_first`, `dropout` (entre capas si `num_layers>1`), `bidirectional`.

nn.LSTM — Variante recurrente con memoria de largo plazo explícita (estado de celda `c_t`) y tres compuertas (`forget`, `input`, `output`) que controlan qué información se olvida, se agrega y se expone como salida. Mitiga el *vanishing gradient* de la RNN simple. Mismos parámetros que `nn.RNN` (`input_size`, `hidden_size`, `num_layers`, `batch_first`, `dropout`, `bidirectional`), sin `nonlinearity` (las compuertas usan `sigmoid/tanh` internamente, no configurable).

nn.utils.rnn.pad_sequence — Recibe una lista de tensores 1D de longitud variable y las apila en un único tensor rellenando con `padding_value` (típicamente 0) hasta la longitud de la secuencia más larga del batch. Parámetros: `sequences`, `batch_first`, `padding_value`.

pack_padded_sequence — Comprime un tensor ya paddeado (junto con las longitudes reales de cada secuencia) en un `PackedSequence`, permitiendo que `nn.RNN/nn.LSTM` procesen solo los pasos de tiempo reales de cada secuencia (evita cómputo y gradiente sobre el padding), siendo además más eficiente. Parámetros: `input`, `lengths`, `batch_first`, `enforce_sorted` (si `False`, no exige batch ordenado por longitud descendente).

pad_packed_sequence — Operación inversa: convierte un `PackedSequence` de vuelta a un tensor denso con padding, útil si se necesita la salida en cada paso de tiempo (arquitecturas many-to-many). No se usa en este laboratorio (arquitectura many-to-one, solo se usa el último `h_n`), pero se investigó por completitud. Parámetros: `sequence`, `batch_first`, `padding_value`, `total_length`.

torch.nn.utils.clip_grad_norm — Recorta la norma global de los gradientes de todos los parámetros de un modelo a un valor máximo (`max_norm`), *in-place*, justo antes de `optimizer.step()`. Se usa para controlar el *exploding gradient*, muy común al entrenar RNNs sobre secuencias largas. Parámetros: `parameters`, `max_norm`, `norm_type` (por defecto norma L2).

nn.Dropout en capas recurrentes — Apaga aleatoriamente una fracción de activaciones durante entrenamiento para regularizar. En `nn.RNN/nn.LSTM`, el parámetro `dropout` del constructor aplica dropout **entre capas apiladas** (solo tiene efecto si `num_layers > 1`); para regularizar también con una sola capa, se agrega explícitamente una `nn.Dropout` sobre el estado oculto final antes de la capa de clasificación. Parámetro: `p` (probabilidad de apagar una unidad).

Hidden state vs. cell state (LSTM). El *hidden state* `h_t` es la salida visible de la celda en el paso `t` (lo que se pasa a la siguiente capa o se usa para predecir); es análogo al estado oculto de una RNN simple. El *cell state* `c_t` es una “cinta transportadora” interna de memoria de largo plazo que fluye casi sin transformaciones no lineales entre pasos de tiempo (solo multiplicaciones/sumas moduladas por las compuertas), permitiéndole preservar información por muchos pasos sin que el gradiente se desvanezca. `h_t` se calcula a partir de `c_t` filtrado por la compuerta de salida.

Vanishing / exploding gradient. Al hacer *backpropagation through time*, el gradiente respecto a los parámetros de pasos de tiempo lejanos se calcula como un producto de muchos jacobianos (uno por paso de tiempo). Si las normas de esos jacobianos son sistemáticamente menores a 1, el producto tiende a 0 exponencialmente (*vanishing gradient*): la red deja de poder aprender dependencias de largo plazo. Si son mayores a 1, el producto crece exponencialmente (*exploding gradient*), causando actualizaciones de peso inestables o NaN. El efecto es multiplicativo en la cantidad de pasos de tiempo, por lo que empeora exponencialmente con secuencias más largas.

Cómo las compuertas de la LSTM mitigan el vanishing gradient. La compuerta de *forget* controla, paso a paso, cuánta memoria previa se conserva; cuando se aprende a mantenerla cercana a 1, el camino del gradiente a través de `c_t` se vuelve casi aditivo (suma de contribuciones en vez de un producto largo de jacobianos con `tanh/sigmoid`), permitiendo que el gradiente fluya hacia atrás muchos pasos sin desvanecerse tan agresivamente. Las compuertas de *input* y *output* controlan qué nueva información entra a la memoria y qué parte se expone como

salida en cada paso, dándole a la red control explícito sobre qué recordar/olvidar/exponer, en vez de reescribir todo el estado oculto en cada paso como hace una RNN simple.

4. Construcción y entrenamiento de las arquitecturas

Las tres arquitecturas ya están definidas y programadas en el notebook (`MLPClassifier` y `RecurrentClassifier` con `cell="rnn"/cell="lstm"`), junto con las utilidades compartidas de entrenamiento/evaluación (`run_epoch`, `train_model`, `run_config`). Todas usan la misma capa `nn.Embedding` (entrenada desde cero) y la misma cabeza de clasificación lineal.

Nota de estado: al momento de generar este PDF, la ejecución del notebook llegó hasta completar la rejilla de 5 iteraciones del **MLP**. Las rejillas de la **RNN** y la **LSTM** (5 configuraciones cada una, ya definidas en el código, variando `hidden_size`, `num_layers`, `dropout` y `gradient clipping`), la selección de mejor configuración, la evaluación final sobre test y las matrices de confusión **todavía no se han ejecutado**, por lo que esas partes se dejan en blanco a continuación.

4.1 MLP — resultados de las 5 iteraciones (EPOCHS=6, BATCH_SIZE=64, MAX_LEN=256)

Configuración	Hiperparámetros	Params	Mejor epoch	Val F1 (macro)	Tiempo
baseline	hidden=[128], dropout=0.3, lr=1e-3	2,013,186	5/6	0.8648	66.9 s
más profundo	hidden=[256,64], dropout=0.3, lr=1e-3	2,042,434	5/6	0.8639	57.4 s
sin dropout	hidden=[128], dropout=0.0, lr=1e-3	2,013,186	5/6	0.8644	46.3 s
lr alto	hidden=[128], dropout=0.3, lr=1e-2	2,013,186	1/6	0.8768	45.5 s
weight decay	hidden=[128], dropout=0.3, lr=1e-3, wd=1e-4	2,013,186	6/6	0.8666	50.6 s

La configuración con `lr=1e-2` obtuvo el mejor F1 de validación (0.8768) pero en la primera época; a partir de la época 2 el `val_loss` sube de forma sostenida (0.2920 → 0.9276 en la época 6) mientras el `train_loss` sigue bajando, una señal clara de **overfitting** por tasa de aprendizaje alta. Las demás configuraciones muestran un patrón más estable, con el mínimo de `val_loss` alrededor de la época 5-6.

(El notebook calcula accuracy/precision/recall/F1 por época en `run_epoch`, pero el log impreso que quedó guardado solo reporta F1 de validación; accuracy/precision/recall por iteración quedarán pendientes de reportar hasta volver a ejecutar la celda con el guardado completo de métricas.)

Gráficas de curvas de pérdida (train/val), RNN y LSTM (al menos 3 iteraciones por arquitectura):

(Pendiente — requiere ejecutar las celdas de entrenamiento de RNN y LSTM, actualmente sin ejecutar.)

Evaluación final sobre test (accuracy, precision, recall, F1, matriz de confusión) de los tres modelos:

(Pendiente — requiere seleccionar la mejor configuración de cada arquitectura y correr la evaluación única sobre test, sección aún sin ejecutar en el notebook.)

4.1 Experimento adicional: efecto de la longitud de secuencia

(Pendiente — el notebook ya tiene programado el experimento con `SEQ_LENS = [50, 450]` para RNN y LSTM, pero no se ha ejecutado.)

5. Comparación de arquitecturas

(Pendiente — depende de los resultados de la evaluación final sobre test de las tres arquitecturas, sección 4, que aún no se ha ejecutado.)

- Cantidad de parámetros entrenables de cada modelo: (pendiente para RNN/LSTM; MLP ronda $\sim 2.0\text{-}2.04M$ según configuración, ver tabla de la sección 4)
- Desempeño sobre test: (pendiente)
- Relación parámetros vs. calidad de resultados: (pendiente)
- Tiempo de entrenamiento aproximado: (para MLP, entre 45-67 s por configuración de 6 épocas en CPU; RNN/LSTM pendiente)
- Resultados del experimento de longitud de secuencia (4.1): (pendiente)

6. Discusión y análisis

- ¿Qué cambio de hiperparámetro tuvo el mayor impacto positivo/negativo en las métricas de validación de cada arquitectura? (Para el MLP: subir el lr a $1e-2$ dio el mejor pico de F1 pero también el mayor impacto negativo, al inducir overfitting agresivo desde la época 2. Para RNN/LSTM: pendiente.)
- ¿La regularización (dropout, weight decay, gradient clipping) mejoró el desempeño en validación? (Para el MLP: $weight_decay=1e-4$ dio el mejor resultado estable sin overfitting -F1=0.8666 en la época 6-, ligeramente mejor que el baseline con dropout y que la variante sin dropout; pendiente de comparar con RNN/LSTM, donde además aplica gradient clipping.)
- Comparando MLP, RNN y LSTM en test: (pendiente)
- Efecto de la longitud de secuencia (4.1) en RNN vs. LSTM y relación con vanishing gradient: (pendiente)
- Tipos de reseñas donde más se equivocan los modelos (largas, con sarcasmo, sentimiento mixto): (pendiente — requiere matrices de confusión y ejemplos mal clasificados de la sección 4)
- Arquitectura recomendada para producción (exactitud vs. eficiencia): (pendiente)

Enlace al repositorio: pendiente de completar antes de la entrega.